NAI Project Report: P22

Reinforcement Learning-Based Adaptive Routing for Synchronized AI Workloads in Leaf-Spine Fabrics

Amartya Singh (2022062), Shubhi Jain (2023517)

February 11, 2026

1 Executive Summary

The rapid proliferation of Large Language Models (LLMs) and generative AI has shifted the bottleneck of distributed computing from the processor to the network. Modern AI training workloads, such as Llama-3 or GPT-4, rely on a communication paradigm known as Bulk Synchronous Parallel (BSP). In this model, thousands of GPUs compute gradients in parallel and then synchronize them using collective communication primitives like All-Reduce. These synchronization phases generate massive, highly synchronized bursts of data known as "Elephant Flows."

Traditional data center fabrics utilize Equal-Cost Multi-Path (ECMP) routing to distribute traffic across parallel paths. ECMP relies on static hashing of packet headers (the 5-tuple). While effective for standard web traffic consisting of millions of small "Mice Flows," ECMP is fundamentally "blind" to link utilization. In AI workloads, where the number of flows is small but the bandwidth demand per flow is enormous, static hashing frequently leads to hash collisions. These collisions result in multiple Elephant Flows being pinned to a single link, causing severe congestion and packet drops, while adjacent parallel links remain entirely idle.

This project proposes a Reinforcement Learning (RL) based Adaptive Routing Controller designed to replace the static decision-making of ECMP with an intelligent, telemetry-driven control plane. By leveraging real-time link utilization data and the programmable nature of the SONiC/FRRouting stack, we develop an agent capable of predicting congestion before it leads to packet retransmissions.

Key Expected Contributions:

- **Systemic Integration:** A closed-loop architecture that bridges a Python-based RL agent with a containerized routing engine using industry-standard BGP attributes.
- **Stability Improvement:** A target reduction of at least 10% in flow completion variance and a significant decrease in TCP retransmissions compared to baseline ECMP.
- **Scalability Analysis:** A comparison of heuristic-based threshold triggers versus Proximal Policy Optimization (PPO) models under varying AI traffic intensities.

- **Resource-Optimized Simulation:** A framework for conducting high-fidelity networking research on commodity hardware by utilizing lightweight Dockerized FRR images.

2 Technology Background

2.1 Core Concepts and Definitions

To understand the proposed solution, one must define the foundational elements of modern data center networking:

1. **Clos / Leaf-Spine Topology:** A two-tier architecture providing non-blocking fabric where any two nodes are exactly three hops apart, ensuring predictable latency and massive bisection bandwidth.

2. **BGP (Border Gateway Protocol):** The routing protocol of the internet adapted for data centers. We use eBGP to manage path reachability and influence traffic via "Weights."

3. **Flow-Based Hashing:** The mechanism behind ECMP. It hashes packet headers to ensure packets of the same flow follow the same path, preventing TCP out-of-order delivery issues.

2.2 Use Cases and KPIs

The primary use case is the **AI Data Center**, where the network is part of the compute fabric.

- **Throughput (Mbps):** The effective data rate. Higher throughput directly reduces training time.

- **Link Utilization Variance:** A measure of load balance. High variance indicates a bottleneck.

2.3 Literature Survey

1. **Hedera (2010):** Introduced centralized flow scheduling, proving that detecting "Elephants" and manually assigning them to paths doubles efficiency over ECMP.

2. **CONGA (2014):** A hardware-based solution for congestion-aware load balancing using custom ASIC headers.

3. **Stampa et al. (2017):** Demonstrated that Deep Q-Networks (DQN) can learn topologies and optimize link weights to minimize delay.

4. **FRRouting Project:** The core routing engine for the SONiC OS, providing the BGP multipath levers used in this project.

5. **Schulman et al. (2017):** Introduced PPO, the current gold standard for stable RL training in stochastic environments.

2.4 Current Challenges and Limitations

The primary challenge is **Flow Stickiness**. Standard TCP flows cannot be moved mid-stream without causing reordering. Our project optimizes BGP Weights during compute phases to prepare for the next synchronization burst.

3 Problem Framing

3.1 Specific Problem Addressed

We are addressing the static hashing collision and flow stickiness in AI fabrics. *Evidence of the Problem:* Our baseline tests showed that when a 1Gbps transfer begins, ECMP pins it to a single Spine. If a second flow hashes to the same Spine, that link saturates while the other Spine remains at 0% utilization. Preliminary data showed Spine-1 reaching 1118.12 Mbps (saturation) while Spine-2 remained at 0.00 Mbps, effectively wasting 50% of bisection bandwidth.

3.2 Scope and Assumptions

- **Base Case:** 4 GPUs (Hosts) over a 2-Spine, 2-Leaf fabric.
- **Hardware Constraints:** Due to full SONiC-VS RAM requirements (4GB/node), we use **FRRouting Docker images (70MB/node)**. This allows us to simulate large topologies on a 16GB laptop while maintaining control-plane logic fidelity.

3.3 Timeliness and Importance

As AI clusters scale toward 100,000 GPUs, "Tail Latency" becomes the primary factor in training time. Standard static protocols are no longer sufficient for the bursty, synchronized nature of these workloads. Adaptive routing is timely as the industry moves toward "Self-Optimizing" AI infrastructure.

3.4 RL Agent Architecture & Implementation

The core of our solution is a closed-loop controller that functions as follows:

3.4.1 Observation Space (State)

The agent monitors the state S_t at each time step, defined as:

$$S_t = [u_{eth1}, u_{eth2}, \Delta u_{eth1}, \Delta u_{eth2}] \tag{1}$$

Where u_i is the current utilization (Mbps) of the i^{th} Spine uplink and Δu is the rate of change, allowing the agent to predict pending congestion.

3.4.2 Action Space & Actuator

The agent selects an action A_t from a discrete set of BGP Weight adjustments. The **Actuator** is a Python script using the *Paramiko* library to SSH into the Leaf switch and execute *vtysh* commands to change weights. By increasing the weight of an idle neighbor, the agent forces new incoming Elephant flows onto the under-utilized Spine.

3.4.3 Reward Function

The agent is incentivized to maximize throughput while maintaining balance:

$$R = \sum_{i=1}^{n} Throughput_i - \lambda \cdot |u_{eth1} - u_{eth2}| \tag{2}$$

Where λ is a penalty coefficient for link imbalance.

3.5 Approach and Methodology

Plan A: PPO Control - We utilize Proximal Policy Optimization to handle the bursty nature of AI traffic.
Plan B: Heuristic Fallback - A "High-Watermark" trigger. If any link exceeds 75% utilization while the alternative is idle(less than 10% usage), a weight update is forced.

3.6 Expected KPIs

We target a stability improvement of >10%. Specifically, reducing variance across Spines should decrease TCP Retransmissions, which reached 26,000 in our unoptimized baseline.

4 References

1. Al-Fares, M., et al. "Hedera: Dynamic Flow Scheduling for Data Center Networks." NSDI, 2010.

2. Alizadeh, M., et al. "CONGA: Distributed Congestion-Aware Load Balancing." SIGCOMM, 2014.

3. Stampa, G., et al. "A Deep Reinforcement Learning Approach for Software-Defined Networking." IEEE, 2017.

4. FRRouting Project. "FRR Documentation," `https://frrouting.org/`.

5. Schulman, J., et al. "Proximal Policy Optimization Algorithms." arXiv:1707.06347, 2017.